Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

ПРАКТИЧЕСКАЯ РАБОТА

№ 3

Дисциплина(модуль)/предмет МДК 03.01 Сопровождение и обслуживание программного обеспечения компьютерных систем

Тема: Аудит релиза и метрики надёжности

ВАРИАНТ № 3 — Climate Control

Студент (ка)/учащийся Бойцов Даниил Антонович / _____ /
Ф. И. О. подпись

Группа / Класс **И13-24-11**

Руководитель: **Александра Анатольевна Чудинова** /_____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «05» ноября 2026 г.

Работа аттестована с оценкой _____
прописью

Санкт-Петербург
2026г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 СЦЕНАРИЙ СБОЯ И АУДИТ РЕЛИЗА 4
2 ДОКУМЕНТ POST MORTEM 5
3 РАСЧЁТ ПОКАЗАТЕЛЕЙ НАДЁЖНОСТИ 6
4 ПРОВЕРКА РАСЧЁТА НА ПРОГРАММНОМ СТЕНДЕ 7
ЗАКЛЮЧЕНИЕ 8
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 9
ПРИЛОЖЕНИЕ А ЛИСТИНГИ ИСХОДНОГО КОДА 10
ПРИЛОЖЕНИЕ Б СКРИНШОТЫ 12

ВВЕДЕНИЕ

Практическая работа посвящена теме «Аудит релиза и метрики надёжности» для модуля Climate Control АИС «Умный кампус», вариант 3. Система принимает температуру и влажность от IoT-датчиков. Объект рассмотрения — программные средства приёма, хранения и сопровождения телеметрии.

В отчёте представлены решения для своего варианта, ссылки на исходники, результаты программных проверок и ограничения стенда. Листинги помещены в приложении А, действительные скриншоты — в приложении Б. Проект не подключён к физическому отоплению или вентиляции.

Материалы практической работы в репозитории:
https://github.com/daniil179/mdk0301-climate-control/tree/main/docs/pr03

1 СЦЕНАРИЙ СБОЯ И АУДИТ РЕЛИЗА

Цель работы — провести аудит релизного процесса, оформить Post-Mortem и рассчитать MTBF, MTTR и доступность Climate Control. Инцидент и журнал наблюдения смоделированы для расчёта; это не сведения о реальном сбое кампуса.

Сценарий INC-CC-003: при выпуске 1.1.0 разработчик изменил общий DTO чтения. Вместо поля temperatureC старый клиент получил вложенный объект temperature. API отвечал HTTP 200 и /health был доступен, но клиент не мог прочитать показания. Влияние — недоступность представления климата для старых клиентов, приоритет P1. Приём сообщений продолжался; подтверждённой потери сохранённых данных в модели нет.

Сведения по теме «результаты аудита релиза» представлены в таблице 1.

Таблица 1 — Результаты аудита релиза

Элемент аудита	Обнаруженный пробел	Требуемый барьер
Контракт	В v1 использован DTO новой версии.	Раздельные маршруты и DTO v1/v2.
Проверки	Проверялся HTTP 200 без чтения тела старым клиентом.	Строгая десериализация v1 и проверка значений.
Выпуск	Нет этапа canary с предыдущим клиентом.	Ограниченный запуск и проверка ключевого сценария.
Наблюдение	Health не отражал ошибку потребителя.	Мониторинг успешности чтения и давности данных.
Возврат	Не проверена совместимость приложения с новой схемой.	Аддитивные миграции и проверенный план возврата бинарной версии.

Источник: составлено автором для Climate Control варианта 3

Рекомендация аудита: не разрешать выпуск при отсутствии контракта, миграционного теста, проверенного резервирования и владельца решения. Разбор ориентирован на системные причины, а не на поиск виновного [1].

2 ДОКУМЕНТ POST MORTEM

Сведения по теме «хронология учебного инцидента» представлены в таблице 2.

Таблица 2 — Хронология учебного инцидента

Момент	Событие и действие
T+00 мин	Развёрнута версия 1.1.0 с изменённым DTO.
T+05 мин	Получен сигнал об ошибке чтения старым клиентом.
T+10 мин	Объявлен P1, назначен координатор, новые выкладки остановлены.
T+20 мин	Сравнение ответов выявило несовместимый контракт.
T+30 мин	Возвращён совместимый бинарный релиз без обратной миграции БД.
T+40 мин	Canary-проверка старого клиента подтверждает чтение показаний.
T+45 мин	Доступность подтверждена, инцидент переведён в наблюдение.

Источник: составлено автором для Climate Control варианта 3

Корневая причина: политика версионирования не была обеспечена тестом старого потребителя. Цепочка «5 почему»: показания не открывались → клиент не прочитал JSON → структура v1 была изменена → DTO разделялся между версиями → в критериях выпуска отсутствовала независимая контрактная проверка.

Сведения по теме «корректирующие мероприятия» представлены в таблице 3.

Таблица 3 — Корректирующие мероприятия

Мера	Ответственный и срок от разбора	Проверка закрытия
Разделить DTO и маршруты	Разработчик, D+1	V01 читает v1, V02 подтверждает v2.
Добавить старый клиент в CI	Инженер сборки, D+2	Сломанный контракт блокирует сборку.
Контроль canary и возврата	Инженер сопровождения, D+3	Журнал пробного выпуска и возврата версии.
Дополнить инструкцию L1/L2	Координатор поддержки, D+2	По карточке определяется масштаб и контрактная ошибка.

Источник: составлено автором для Climate Control варианта 3

3 РАСЧЁТ ПОКАЗАТЕЛЕЙ НАДЁЖНОСТИ

Наблюдение: 30 суток непрерывной услуги, T = 720 ч. Три непересекающихся отказа длились 20, 35 и 45 мин. Отказ учитывается от потери целевого пользовательского сценария до подтверждённого восстановления, а не только от остановки процесса API.

Сведения по теме «расчёт метрик надёжности» представлены в таблице 4.

Таблица 4 — Расчёт метрик надёжности

Показатель	Формула и расчёт	Результат
Суммарный простой	D = 20 + 35 + 45	100 мин = 1,666667 ч
Работоспособное время	U = 720 − 100 / 60	718,333333 ч
MTBF	U / N = 718,333333 / 3	239,444444 ч
MTTR	D / N = 100 / 3	33,333333 мин
Доступность	A = U / T × 100%	99,768519%

Источник: составлено автором для Climate Control варианта 3

Применён MTBF как средняя наработка на отказ, поэтому в числителе исключён ремонт. Деление 720 / 3 = 240 ч даёт средний цикл с восстановлением и не подменяет выбранное определение. Для согласованных единиц выполняется A = MTBF / (MTBF + MTTR в часах). При нуле отказов MTBF не оценивается по этой выборке, а не объявляется бесконечным.

Расчёт выполнен в ReliabilityMetrics.cs и повторён короткой программой C# в OneCompiler. Машиночитаемый результат находится в evidence/final/reliability.json. Это проверка арифметики на модельных данных, а не измерение реальной доступности.

План исправления рассматривается как часть жизненного цикла ПО [2]. Контрактный барьер реализован интеграционными тестами ASP.NET Core [3].

4 ПРОВЕРКА РАСЧЁТА НА ПРОГРАММНОМ СТЕНДЕ

Расчёт дополнительно выполнен общей функцией ReliabilityMetrics.Calculate. Для периода 720 часов и простоев 20, 35 и 45 минут получено MTBF = 239,444444 ч, MTTR = 33,333333 мин, доступность = 99,768519 %. MTBF здесь равен времени исправной работы, делённому на число отказов; сумма простоев исключена из числителя. Исходные значения являются моделью учебного инцидента. Результат JSON и запуск C# показаны на рисунках Б.1 и Б.2.

ЗАКЛЮЧЕНИЕ

Выявлены пять пробелов релизного процесса, оформлены хронология и корневая причина сбоя, назначены корректирующие меры. На заданном журнале рассчитаны MTBF 239,444444 ч, MTTR 33,333333 мин и доступность 99,768519%. Результат расчёта воспроизводим.

Материалы относятся к модулю Climate Control варианта 3. Проверенный программный стенд не подключён к физическому отоплению или вентиляции кампуса. Промышленная приёмка оборудования в рамках работы не проводилась.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Google. Site Reliability Engineering. Postmortem Culture Learning from Failure. — URL: https://sre.google/sre-book/postmortem-culture/ (дата обращения: 11.11.2026).
2. ISO. ISO/IEC/IEEE 12207:2026. Software life cycle processes. — URL: https://www.iso.org/standard/90219.html (дата обращения: 11.11.2026).
3. Microsoft Learn. Integration tests in ASP.NET Core. — URL: https://learn.microsoft.com/en-us/aspnet/core/test/integration-tests?view=aspnetcore-8.0 (дата обращения: 11.11.2026).

ПРИЛОЖЕНИЕ А ЛИСТИНГИ ИСХОДНОГО КОДА

Ниже приведены фрагменты реальных исходников. Для сборки используется всё решение, размещённое в общем репозитории. Полные файлы доступны по ссылкам; номера строк относятся к опубликованной версии.

Общее решение: https://github.com/daniil179/mdk0301-climate-control/tree/main/projects/ClimateControl

src/Climate.Operations/ReliabilityMetrics.cs

tests/Climate.UnitTests/OperationsTests.cs

Листинг А.1 — Расчёт MTBF и MTTR

Исходный файл: ReliabilityMetrics.cs

```
   public static ReliabilityResult Calculate(double observationHours, params
double[] downtimeMinutes)
   {
      if (observationHours <= 0 || downtimeMinutes.Any(x => x < 0) ||
downtimeMinutes.Sum() > observationHours * 60)
         throw new ArgumentOutOfRangeException(nameof(observationHours));
      double down = downtimeMinutes.Sum();int n = downtimeMinutes.Length;
      double uptime = observationHours - down / 60;
      return new(observationHours, down, n, n == 0 ? null : uptime / n,
         n == 0 ? null : down / n, uptime / observationHours * 100);
   }
```

ПРИЛОЖЕНИЕ Б СКРИНШОТЫ

На рисунке Б.1 показан фрагмент: Учебный расчёт метрик в среде исполнения C#.

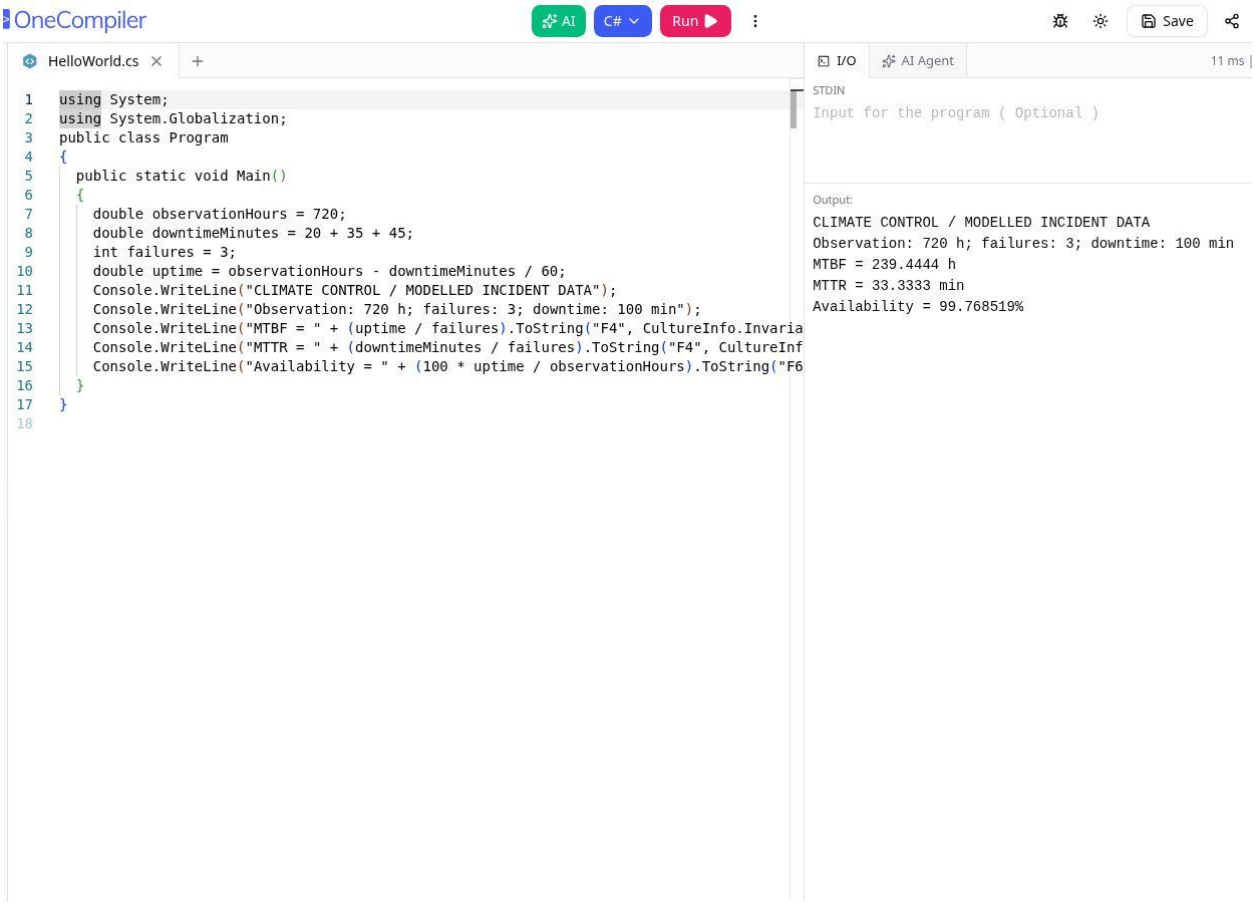


Рисунок Б.1 — Учебный расчёт метрик в среде исполнения C#

На рисунке Б.2 показан фрагмент: Результат повторного расчёта в JSON.

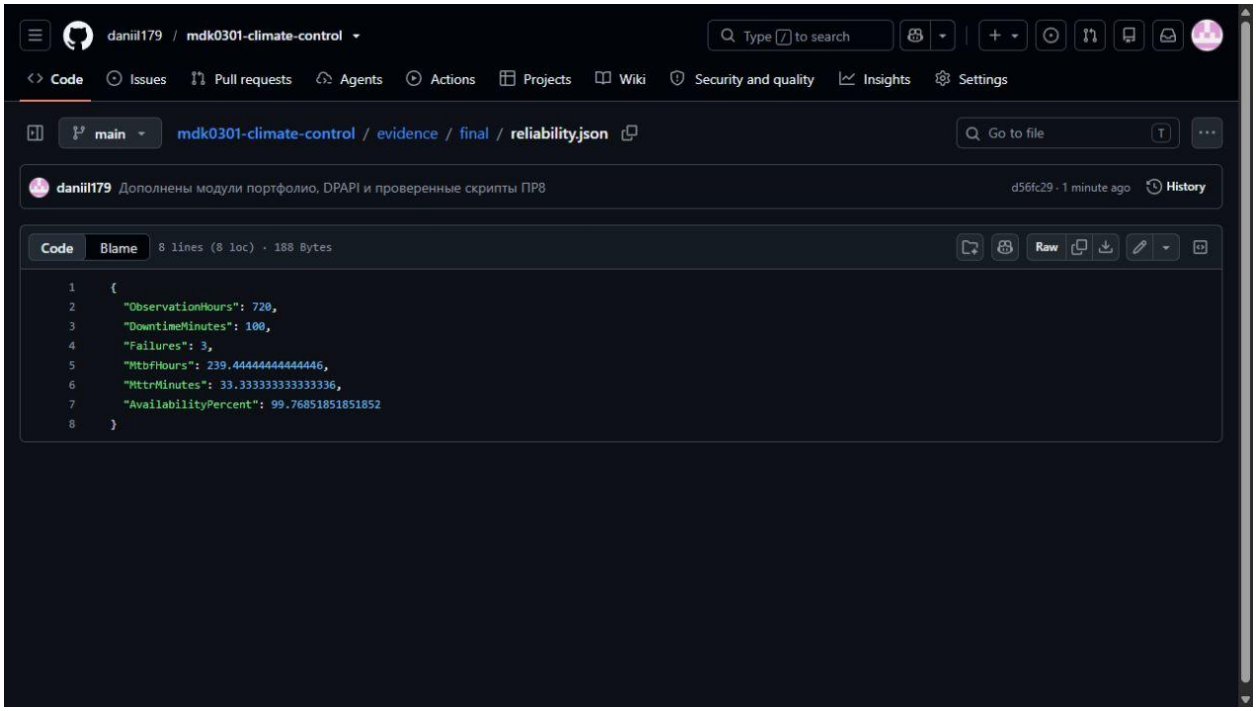


Рисунок Б.2 — Результат повторного расчёта в JSON